

Лабораторная работа №2

Компьютерный практикум по статистическому анализу данных

Лисенков Е.Р.

Российский университет дружбы народов, Москва, Россия

Информация

- Лисенков Егор Романович
- студент НФИбд-02-23
- Российский университет дружбы народов
- 1132232881@rudn.ru
- <https://github.com/erlisenkov>



Ц

Вводная часть

Вводная часть

Структуры данных являются основой эффективного программирования на языке Julia, и их освоение необходимо для решения задач статистического анализа данных.

Объектом выступают структуры данных языка Julia, а предметом — операции создания, индексации, преобразования и комбинирования этих структур.

Целью работы является изучение кортежей, словарей, множеств и массивов, а задачами — воспроизведение примеров методических указаний и выполнение заданий самостоятельной работы в среде Jupyter Lab.

Работа выполнена в операционной системе Fedora с использованием языка Julia версии 1.10.3, интерактивной среды Jupyter Lab и системы контроля версий Git.

Запуск сервера Jupyter Lab из каталога лабораторной работы открыл интерактивную среду для выполнения вычислений на языке Julia.

```
egorlisenkov@fedora:~$ cd ~/komputerni-praktikum-2026-2027
egorlisenkov@fedora:~/komputerni-praktikum-2026-2027$ cd labs/lab02
egorlisenkov@fedora:~/komputerni-praktikum-2026-2027/labs/lab02$ jupyter lab
[I 2026-09-26 14:15:56.980 ServerApp] Package jupyterlab took 0.0000s to import
[I 2026-09-26 14:15:57.048 ServerApp] Package jupyter_lsp took 0.0654s to import
[W 2026-09-26 14:15:57.048 ServerApp] A `_jupyter_server_extension_points` function was not
found in jupyter_lsp. Instead, a `_jupyter_server_extension_paths` function was found and
will be used for now. This function name will be deprecated in future releases of jupyter_lsp.
```

Блокнот и общие функции коллекций

В созданном блокноте продемонстрирована разница режимов Code и Markdown и проверены общие функции коллекций `isEmpty`, `length`, `in`, `unique`, `reduce`, `maximum` и `minimum`.

```
# Лабораторная работа 2. Структуры данных
Выполнил: Лисенков Е.Р.
```

```
UndefVarError: 'Выполнил' not defined
```

```
Stacktrace:
 [1] top-level scope
 @ In[1]:2
```

```
v = [1, 2, 3, 3]
println("isEmpty: ", isEmpty(v))
println("length: ", length(v))
println("in: ", in(2, v), " и ", 0 in v)
println("unique: ", unique(v))
println("reduce(+): ", reduce(+, v))
println("maximum / minimum: ", maximum(v), " / ", minimum(v))
```

```
isEmpty: false
length: 4
in: true и false
unique: [1, 2, 3]
reduce(+): 9
maximum / minimum: 3 / 1
```

На примерах кортежей и словарей показаны доступ к элементам по индексу и по ключу, добавление и удаление пар, а также слияние словарей с разрешением конфликта ключей.

```
maximum / minimum: 3 / 1

[: t_empty = ()
 favoritelang = ("Python", "Julia", "R")
 x1 = (1, 2, 3)
 x2 = (1, 2.0, "tmp")
 x3 = (a = 2, b = 1 + 2)

println("length(x2) = ", length(x2))
println("элементы x2: ", x2[1], ", ", x2[2], ", ", x2[3])
c = x1[2] + x1[3]
println("c = x1[2] + x1[3] = ", c)
println("x3.a, x3.b, x3[2] = ", x3.a, ", ", x3.b, ", ", x3[2])
println("in(\"tmp\", x2) = ", in("tmp", x2), "; 0 in x2 = ", 0 in x2)

length(x2) = 3
элементы x2: 1, 2.0, tmp
c = x1[2] + x1[3] = 5
x3.a, x3.b, x3[2] = 2, 3, 3
in("tmp", x2) = true; 0 in x2 = false

[: phonebook = Dict{"Иванов И.И." => {"867-5309", "333-5544"},
 "Бухгантерия" => "555-2368"}
println("keys: ", keys(phonebook))
println("values: ", values(phonebook))
println("pairs: ", pairs(phonebook))
println("haskey: ", haskey(phonebook, "Иванов И.И."))

phonebook["Сидоров П.С."] = "555-3344"
println(phonebook)
pop!(phonebook, "Иванов И.И.")
println(phonebook)

a = Dict{"foo" => 0.0, "bar" => 42.0}
b = Dict{"baz" => 17, "bar" => 13.0}
println("merge(a,b): ", merge(a, b))
println("merge(b,a): ", merge(b, a))

keys: ["Бухгантерия", "Иванов И.И."]
values: Any["555-2368", ("867-5309", "333-5544")]
pairs: Dict{String, Any}{"Бухгантерия" => "555-2368", "Иванов И.И." => ("867-5309", "333-5544")}
haskey: true
Dict{String, Any}{"Сидоров П.С." => "555-3344", "Бухгантерия" => "555-2368", "Иванов И.И." => ("867-5309", "333-5544")}
Dict{String, Any}{"Сидоров П.С." => "555-3344", "Бухгантерия" => "555-2368"}
merge(a,b): Dict{String, Real}{"bar" => 13.0, "baz" => 17, "foo" => 0.0}
merge(b,a): Dict{String, Real}{"bar" => 42.0, "baz" => 17, "foo" => 0.0}
```

Операции над множествами и создание массивов различной размерности подтвердили семантику математических множеств и гибкость конструкторов массивов.

```
Dict{String, Any}{"Супероп Н.С." => "555-3344", "Бухгалтерия" => "555-2368"}
merge(a,b): Dict{String, Real}{"bar" => 13.0, "baz" => 17, "foo" => 0.0}
merge(b,a): Dict{String, Real}{"bar" => 42.0, "baz" => 17, "foo" => 0.0}
```

```
A = Set{1, 3, 4, 5}
B = Set{"abracadabra"}
println("A = ", A, "; B = ", B)

S1 = Set{1, 2}; S2 = Set{3, 4}
println("isetequal(S1,S2) = ", isetequal(S1, S2))
S3 = Set{1, 2, 3, 1, 2, 3, 2, 1}; S4 = Set{2, 3, 1}
println("isetequal(S3,S4) = ", isetequal(S3, S4))

println("union: ", union(S1, S2))
println("intersect: ", intersect(S1, S3))
println("setdiff: ", setdiff(S3, S1))
println("issubset(S1,S4): ", issubset(S1, S4))
```

```
push!(S4, 99); println("nocne push!: ", S4)
pop!(S4); println("nocne pop!: ", S4)
```

```
A = Set{5, 4, 3, 1}; B = Set{'a', 'd', 'r', 'k', 'b'}
isetequal(S1,S2) = false
isetequal(S3,S4) = true
union: Set{4, 2, 3, 1}
intersect: Set{2, 1}
setdiff: Set{3}
issubset(S1,S4): true
nocne push!: Set{2, 99, 3, 1}
nocne pop!: Set{99, 3, 1}
```

```
empty_array_1 = []
empty_array_2 = {Int64}[]
empty_array_3 = {Float64}[]
```

```
a = [1, 2, 3]
b = [1 2 3]
Amat = [[1 2 3]; [4 5 6]; [7 8 9]]
```

```
c_rand = rand(1, 8)
C_rand = rand(2, 3)
D_rand = rand(4, 3, 2)
```

```
roots = [sqrt(i) for i in 1:10]
ar_1 = [3*i^2 for i in 1:2:9]
ar_2 = [i^2 for i = 1:10 if (i^2 % 5 != 0 && i^2 % 4 != 0)]
```

```
(a, b, Amat, roots, ar_1, ar_2)
```

```
{1, 2, 3}, {1 2 3}, {1 2 3; 4 5 6; 7 8 9}, [1.0, 1.4142135623730951, 1.7320508075688772, 2.0, 2.23606797749979, 2.449489742783178, 2.6457513110645907, 2.8284271247461903, 3.0, 3.1622776601683795], [3, 27, 75, 147, 243], [1, 9, 49, 81]}
```

Операции над массивами

Для массивов вычислены размерные характеристики, выполнены преобразование формы, транспонирование, срезы и сортировка элементов.

```
Aop = [[1 2 3]; [4 5 6]; [7 8 9]]
println("length, ndims, size, size(A,1): ",
        length(Aop), " ", ndims(Aop), " ", size(Aop), " ", size(Aop, 1))
println("ones(5): ", ones(5))
println("ones(2,3): ", ones(2, 3))
println("zeros(4): ", zeros(4))
println("fill(3.5, (3,2)):", fill(3.5, (3, 2)))
println("repeat([1,2],3,3): ", repeat([1, 2], 3, 3))
println("repeat([1 2],3,3): ", repeat([1 2], 3, 3))
```

```
acol = collect(1:12)
bmat = reshape(acol, (2, 6))
println("reshape: ", bmat)
println("транспонирование b': ", bmat')
println("transpose(b): ", transpose(bmat))

length, ndims, size, size(A,1): 9, 2, (3, 3), 3
ones(5): [1.0, 1.0, 1.0, 1.0, 1.0]
ones(2,3): [1.0 1.0 1.0; 1.0 1.0 1.0]
zeros(4): [0.0, 0.0, 0.0, 0.0]
fill(3.5, (3,2)): [3.5 3.5; 3.5 3.5; 3.5 3.5]
repeat([1,2],3,3): [1 1 1; 2 2 2; 1 1 1; 2 2 2; 1 1 1; 2 2 2]
repeat([1 2],3,3): [1 2 1 2 1 2; 1 2 1 2 1 2; 1 2 1 2 1 2]
reshape: [1 3 5 7 9 11; 2 4 6 8 10 12]
транспонирование b': [1 2; 3 4; 5 6; 7 8; 9 10; 11 12]
transpose(b): [1 2; 3 4; 5 6; 7 8; 9 10; 11 12]
```

```
using Random
Random.seed!(42)
ar = rand(10:20, 10, 5)
println("массив ar:\n", ar)
println("столбец 2: ", ar[:, 2])
println("столбцы 2 и 5: ", ar[:, [2, 5]])
println("строки 2:4,6 и столбцы 1:5: ", ar[[2, 4, 6], [1, 5]])
println("строка 1, столбцы 3:end: ", ar[1, 3:end])
println("сортировка по столбцам:\n", sort(ar, dims=1))
println("сортировка по строкам:\n", sort(ar, dims=2))
println("ar .> 14:\n", ar .> 14)
println("индексы: ", findall(ar .> 14))
```

```
массив ar:
[16 17 10 13 15; 14 17 15 11 15; 15 13 12 17 19; 17 12 16 14 11; 17 15 14 10 13; 11 10 13 10 20; 16 12 14 17 13; 17 12 19 15 18; 15 14 13 17 18; 13 15 16 11 16]
столбец 2: [17, 17, 13, 12, 15, 10, 12, 12, 14, 15]
столбцы 2 и 5: [17 15; 17 15; 13 19; 12 11; 15 13; 10 20; 12 13; 12 18; 14 18; 15 16]
столбцы 2:4:
[17 10 13; 17 15 11; 13 12 17; 12 16 14; 15 14 10; 10 13 10; 12 14 17; 12 19 15; 14 13 17; 15 16 11]
строки 2,4,6 и столбцы 1:5: [14 15; 17 11; 11 20]
строка 1, столбцы 3:end: [10, 13, 15]
сортировка по столбцам:
[11 10 10 10 11; 13 12 12 10 13; 14 12 13 11 13; 15 12 13 11 15; 15 13 14 13 15; 16 14 14 14 16; 16 15 15 15 18; 17 15 16 17 18; 17 17 16 17 19; 17 17 19 17 20]
сортировка по строкам:
[10 13 15 16 17; 11 14 15 15 17; 12 13 15 17 19; 11 12 14 16 17; 10 13 14 15 17; 10 10 13 13 20; 12 13 14 16 17; 12 15 17 18 19; 13 14 15 17 18; 11 13 15 16 16]
ar .> 14:
Bool[1,1,0,0,1,0,1,1,0,1,1,0,0,1,1,0,0,0,0,0,0,0,1,0,0,1,0,1,0,1,1,1,1,0,0,1,1,1,0,1,1,0,1,1]
```

Операции над множествами в самостоятельной работе

В самостоятельной работе найдено итоговое множество как объединение попарных пересечений и продемонстрированы операции над множествами элементов разных типов.

```
A = Set{10, 3, 4, 9}
B = Set{11, 3, 4, 7}
C = Set{10, 1, 2, 4, 7, 8, 9}
P = union(intersect(A, B), intersect(A, C), intersect(B, C))
println("P = ", P)

P = Set{10, 4, 7, 9, 3, 11}
```

```
M1 = Set{1, 2.5, "Julia", 'c'}
M2 = Set{2.5, "Julia", (1, 2)}
println("union: ", union(M1, M2))
println("intersect: ", intersect(M1, M2))
println("setdiff: ", setdiff(M1, M2))
println("issubset: ", issubset(intersect(M1, M2), M1))

union: Set{Any{1, 2}, 'c', "Julia", 2.5, 1}
intersect: Set{Any{"Julia", 2.5}}
setdiff: Set{Any{1, 2}}
issubset: true
```

Массивы заданной структуры

Массивы заданной структуры построены с помощью функций collect, fill и repeat, а в полученном векторе подсчитано число вхождений шестёрки.

```
N = 25
arr_31 = collect(1:N)
arr_32 = collect(N:-1:1)
arr_33 = [collect(1:N); collect(N-1:-1:1)]
println(arr_31); println(arr_32); println(arr_33)

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25]
[25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

```
tmp = [4, 6, 3]
arr_35 = fill(tmp[1], 10)
arr_36 = repeat(tmp, 10)
arr_37 = [fill(tmp[1], 11); fill(tmp[2], 10); fill(tmp[3], 10)]
arr_38 = [repeat([tmp[1]], 10); repeat([tmp[2]], 20); repeat([tmp[3]], 30)]
println(arr_35); println(arr_36); println(arr_37); println("длина arr_38 = ", length(arr_38))

[4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
[4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3]
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 6, 6, 6, 6, 6, 6, 6, 6, 6, 3, 3, 3, 3, 3, 3, 3, 3, 3]
длина arr_38 = 60
```

```
arr_39 = [2*tmp[1], 2*tmp[2], fill(2*tmp[3], 4)...]
count6 = count(==(6), arr_39)
println("массив: ", arr_39)
println("цифра 6 встречается раз: ", count6)

массив: [8, 12, 6, 6, 6, 6]
цифра 6 встречается раз: 4
```

Сгенерированы векторы пар, степенных отношений и строковых имён, а также вычислено среднее значение заданной экспоненциально-тригонометрической зависимости на сетке от трёх до шести.

Сгенерированы векторы пар, степенных отношений и строковых имён, а также вычислено среднее значение заданной экспоненциально-тригонометрической зависимости на сетке от трёх до шести.

```
using Statistics
x10 = 3:0.1:6
y10 = exp.(x10) .* cos.(x10)
println("среднее значение y = ", mean(y10))

среднее значение y = 53.11374594642971
```

```
vec_311 = [(0.1*i, 0.2*j) for (i, j) in zip(3:3:36, 1:3:34)]
M = 25
vec_312 = [2^i / i for i in 1:M]
vec_313 = ["fn$i" for i in 1:30]
println(vec_311); println(vec_312[1:5]); println(vec_313[1:5])
```

```
[(0.30000000000000004, 0.2), (0.6000000000000001, 0.8), (0.9, 1.4000000000000001), (1.2000000000000002, 2.0), (1.5, 2.6), (1.8, 3.2), (2.1, 3.8000000000000003), (2.4000000000000004, 4.4), (2.7, 5.0), (3.0, 5.6000000000000005), (3.3000000000000003, 6.2), (3.6, 6.8000000000000001)]
[2.0, 2.0, 2.6666666666666665, 4.0, 6.4]
["fn1", "fn2", "fn3", "fn4", "fn5"]
```

Анализ случайных выборок

Выполнен статистический анализ случайных выборок длины 250 с преобразованиями векторов, отбором элементов по условию и сортировкой по связанному вектору.

```
using Random, Statistics
Random.seed!(123)
n = 250
x = rand(0.999, n)
y = rand(0.999, n)

v_1 = y[2:end] ./ x[1:end-1]
v_2 = x[1:end-2] .* 2 ./ x[2:end-1] ./ x[3:end]
v_3 = sin.(y[1:end-1]) ./ cos.(x[2:end])
v_sum = sum(exp.(-x[2:end]) ./ (x[1:end-1] ./ 10))
println("длины: ", length(v_1), ", ", length(v_2), ", ", length(v_3))
println("сумма = ", v_sum)

длины: 249, 248, 249
сумма = 0.00017065210565897441

mask = y .> 600
println("элементы y > 600: ", y[mask])
println("их индексы: ", findall(mask))
println("соответствующие x: ", x[mask])

v_4 = sqrt.(abs.(x - mean(x)))
println("в пределах 200 от максимума: ", count(y .> maximum(y) - 200))
println("чётных / нечётных в x: ", count(iseven.(x)), " / ", count(isodd.(x)))
println("кратных 7 в x: ", count(x .% 7 .== 0))
println("x, отсортированный по y (первые 10): ", x[sortperm(y)[1:10]])
println("top-10 элементов x: ", x[sortperm(x, rev=true)[1:10]])
println("уникальных элементов x: ", length(unique(x)), " шт.")

элементы y > 600: [650, 753, 911, 670, 922, 767, 655, 824, 847, 905, 823, 708, 679, 738, 740, 952, 817, 981, 700, 757, 820, 769, 887, 911, 990, 947, 640, 939, 712, 954, 630, 908, 996, 845, 810, 988, 714, 658, 738, 882, 756, 745, 681, 732, 715, 838, 795, 9
60, 867, 628, 950, 719, 846, 829, 877, 852, 670, 975, 799, 711, 601, 697, 766, 822, 770, 882, 690, 913, 640, 675, 609, 684, 895, 956, 853, 944, 689, 642, 805, 915, 650, 627, 812, 694, 743, 907, 880, 655, 709, 804, 751, 756, 655, 607, 894, 853, 727, 872, 6
92, 804, 687, 713, 978, 999, 902, 945, 808, 666, 695, 928, 736, 766, 842, 922]
их индексы: [5, 6, 7, 8, 9, 10, 11, 12, 14, 17, 21, 22, 23, 25, 35, 36, 37, 38, 39, 40, 41, 42, 45, 48, 49, 52, 53, 54, 55, 57, 59, 60, 62, 63, 68, 69, 72, 73, 75, 77, 80, 88, 89, 90, 91, 96, 98, 99, 102, 104, 110, 111, 113, 117, 120, 121, 123, 126, 128,
129, 130, 133, 135, 136, 138, 142, 143, 144, 146, 147, 148, 150, 152, 154, 159, 160, 162, 163, 164, 165, 170, 175, 177, 179, 183, 184, 185, 188, 192, 193, 194, 197, 205, 206, 207, 213, 214, 215, 217, 220, 223, 225, 227, 230, 232, 233, 235, 236, 238, 240,
245, 247, 249, 250]
соответствующие x: [525, 390, 44, 933, 580, 327, 526, 836, 465, 720, 276, 983, 880, 380, 581, 204, 386, 18, 72, 914, 387, 771, 264, 732, 103, 223, 850, 375, 676, 706, 878, 417, 196, 585, 883, 541, 932, 840, 794, 445, 787, 295, 621, 333, 704, 180, 868, 106
, 57, 195, 544, 274, 788, 456, 46, 974, 686, 87, 932, 35, 21, 671, 177, 204, 383, 165, 607, 422, 585, 133, 996, 986, 652, 693, 959, 782, 613, 132, 953, 435, 275, 195, 596, 23, 158, 993, 708, 180, 42, 32, 788, 524, 784, 84, 184, 729, 480, 282, 472, 843, 76
, 487, 95, 858, 664, 834, 463, 321, 136, 200, 222, 2, 391, 885]
в пределах 200 от максимума: 57
чётных / нечётных в x: 129 / 121
кратных 7 в x: 30
x, отсортированный по y (первые 10): [471, 799, 152, 34, 180, 404, 681, 597, 311, 84]
top-10 элементов x: [996, 993, 993, 986, 983, 974, 959, 953, 949, 943]
уникальных элементов x: 227 шт.
```

Квадраты целых чисел и пакет Primes

Создан массив квадратов целых чисел от 1 до 100 и подключён пакет Primes для генерации простых чисел.

```
println("уникальных элементов x: ", length(unique(x)), " шт.")

элементы y > 600: [650, 753, 911, 670, 922, 767, 655, 824, 847, 905, 823, 700, 679, 730, 740, 952, 817, 981, 700, 757, 820, 769, 807, 911, 990, 947, 640, 939, 712, 954, 630, 908, 996, 845, 810, 980, 714, 650, 730, 802, 756, 745, 681, 732, 715, 838, 795, 9
60, 867, 628, 950, 719, 846, 829, 877, 852, 670, 975, 799, 711, 601, 697, 766, 822, 770, 802, 690, 913, 640, 675, 609, 604, 895, 956, 853, 944, 609, 642, 805, 915, 650, 627, 812, 694, 743, 907, 880, 655, 709, 804, 751, 756, 655, 607, 894, 853, 727, 872, 6
92, 884, 687, 713, 970, 999, 902, 945, 808, 666, 695, 920, 736, 766, 842, 922]
их индексы: [5, 6, 7, 8, 9, 10, 11, 12, 14, 17, 21, 22, 23, 25, 33, 36, 37, 38, 39, 40, 41, 42, 45, 48, 49, 52, 53, 54, 55, 57, 59, 60, 62, 63, 68, 69, 72, 73, 75, 77, 80, 88, 89, 90, 91, 96, 98, 99, 102, 104, 110, 111, 113, 117, 120, 121, 123, 126, 128,
129, 130, 133, 135, 136, 138, 142, 143, 144, 146, 147, 148, 150, 152, 154, 159, 160, 162, 163, 164, 165, 170, 175, 177, 179, 183, 184, 185, 188, 192, 193, 194, 197, 205, 206, 207, 213, 214, 215, 217, 220, 223, 225, 227, 230, 232, 233, 235, 236, 238, 240,
245, 247, 249, 250]
соответствующие x: [525, 390, 44, 933, 580, 327, 526, 836, 465, 720, 276, 983, 880, 380, 581, 204, 386, 18, 72, 914, 387, 771, 264, 732, 183, 223, 858, 375, 676, 796, 878, 417, 196, 585, 803, 541, 932, 840, 794, 445, 707, 295, 621, 333, 704, 180, 868, 106
, 57, 195, 544, 274, 788, 456, 46, 974, 606, 87, 932, 35, 21, 671, 177, 204, 383, 165, 607, 422, 585, 133, 996, 986, 652, 693, 959, 782, 613, 132, 953, 435, 275, 195, 596, 23, 150, 993, 700, 180, 42, 32, 788, 524, 704, 84, 184, 729, 480, 202, 472, 843, 76
, 487, 95, 858, 664, 834, 463, 321, 136, 200, 222, 2, 391, 885]
в пределах 200 от максимума: 57
чётных / нечётных в x: 129 / 121
кратных 7 в x: 30
x, отсортированный по y (первые 10): [471, 799, 152, 34, 100, 404, 681, 597, 311, 84]
top-10 элементов x: [990, 993, 993, 986, 983, 974, 959, 953, 949, 943]
уникальных элементов x: 227 шт.
```

```
squares = [i^2 for i in 1:100]
println(squares[1:10], "...")

[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]...
```

```
using Pkg; Pkg.add("Primes")
using Primes
myprimes = primes(1000)
println("конечная: ", length(myprimes))
println("89-е простое: ", myprimes[89])
println("срез 89:99: ", myprimes[89:99])
```

Updating registry at "~/.julia/registries/General.toml"

```
sum_61 = sum(i^3 + 4*i^2 for i in 1:100)
M62 = 25
sum_62 = sum(2*i + 3*i / i^2 for i in 1:M62)

total = 1.0
for k in 2:38
    total += prod(2:k) / prod((k+1):(2k-1))
end
println("6.1 = ", sum_61)
println("6.2 = ", sum_62)
println("6.3 = ", total)
```

Итоговые результаты заданий 4-6

Получены итоговые результаты: 168 простых чисел, 89-е наименьшее простое число 461, срез массива с 89-го по 99-й элемент и вычисленные значения трёх сумм.

```
8): squares = [i^2 for i in 1:100]
println(squares[1:10], "...")
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]...

9): using Pkg; Pkg.add("Primes")
using Primes
myprimes = primes(1000)
println("количество: ", length(myprimes))
println("89-е простое: ", myprimes[89])
println("срез 89:99: ", myprimes[89:99])

Updating registry at '~/.julia/registries/General.toml'
Resolving package versions...
Updating '~/.komputerni-praktikum-2026-2027/Project.toml'
[27ebfcd6] + Primes v0.5.7
Updating '~/.komputerni-praktikum-2026-2027/Manifest.toml'
[18e54dd8] + IntegerMathUtils v0.1.4
[27ebfcd6] + Primes v0.5.7
количество: 168
89-е простое: 461
срез 89:99: [461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523]

0): sum_61 = sum(i^3 + 4*i^2 for i in 10:100)
M62 = 25
sum_62 = sum(2*i + 3*i / i^2 for i in 1:M62)

total = 1.0
for k in 2:38
    total += prod(2:k) / prod((k+1):(2k-1))
end
println("6.1 = ", sum_61)
println("6.2 = ", sum_62)
println("6.3 = ", total)

6.1 = 26852735
6.2 = 2.1934716918435388e9
6.3 = 22.37818348501321
```

1:



В ходе лабораторной работы изучены основные структуры данных языка Julia: кортежи, словари, множества и массивы. Освоены операции создания, индексации, преобразования и комбинирования структур данных, продемонстрирована эффективность генераторов массивов и векторизованных операций для решения прикладных задач. Все задания методических указаний выполнены в полном объёме, а результаты зафиксированы в системе контроля версий Git.